
Optimization

(0) Optimizer 개요

Department of Computer Engineering and Artificial Intelligence
MAI Lab.

YS Park

Model optimization 개요

(1) Optimizer 개요

(1) 학습이란 무엇인가?

(2) backward()

(3) Base class: optim.Optimizer()

(2) (Stochastic) Gradient Descent

(3) Momentum

(4) Adaptive learning rate

(5) Adam

우리는 딥러닝 모델의 파라미터를 최적화 기법을 통해 학습한다.



```
for epoch in range(epochs):  
    for batch_X, batch_y in dataloader:  
        optimizer.zero_grad()          # Clear old gradients  
        outputs = model(batch_X)        # Forward pass  
        loss = criterion(outputs, batch_y) # Calculate loss  
        loss.backward()                  # Backward pass  
        optimizer.step()                 # Update weights
```

딥러닝 모델은 파라미터 θ 로 이루어진 함수 $f_{\theta}(x)$ 로 표기

학습: 파라미터 θ 를 **조정**해서, 모델이 **예측한 값**과 **실제 정답** 사이의 차이(**손실, loss**)를 최소화하는 과정

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(x,y)} [\mathcal{L}(f_{\theta}(x), y)]$$

그 과정을 면밀히 자세히 살펴보면 다음과 같음

$$\underbrace{\hat{y} = f_{\theta}(x)}_{\text{Forward}} \Rightarrow \underbrace{L = \mathcal{L}(\hat{y}, y)}_{\text{Loss}} \Rightarrow \underbrace{\nabla_{\theta} L}_{\text{Backward}} \Rightarrow \underbrace{\theta \leftarrow \theta - \eta \nabla_{\theta} L}_{\text{Update}}$$

딥러닝 모델은 파라미터 θ 로 이루어진 함수 $f_{\theta}(x)$ 로 표기

학습: 파라미터 θ 를 **조정**해서, 모델이 **예측한 값**과 **실제 정답** 사이의 차이(**손실, loss**)를 최소화하는 과정

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(x,y)} [\mathcal{L}(f_{\theta'}(x)y)]$$

그 과정을 면밀히 자세히 살펴보면 다음과 같음

$$\underbrace{\hat{y} = f_{\theta}(x)}_{\text{Forward}} \Rightarrow \underbrace{L = \mathcal{L}(\hat{y}, y)}_{\text{Loss}} \Rightarrow \underbrace{\nabla_{\theta} L}_{\text{Backward}} \Rightarrow \underbrace{\theta \leftarrow \theta - \eta \nabla_{\theta} L}_{\text{Update}}$$

loss.backward(): 손실 $\mathcal{L}$ 를 각 파라미터 θ_i 로 편미분한 값을 전부 계산하라는 명령

autograd: forward 시 자동으로 만들어진 computational graph를 거꾸로 타고 올라가면서 연쇄법칙으로 각 파라미터까지의 미분을 자동 계산

$$\text{backward()} \Rightarrow \nabla_{\theta} L = \left[\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_n} \right]$$

계산된 gradient는 각 parameter (tensor)의 .grad attribute에 누적

- backward()를 두 번 호출하면 그래디언트가 더해짐. 그래서 매 스텝 시작 전에 **.zero_grad()**로 초기화

Base class: optim.Optimizer()

Base class

`class torch.optim.Optimizer(params, defaults)`

[\[source\]](#)

Base class for all optimizers.

⚠ Warning

Parameters need to be specified as collections that have a deterministic ordering that is consistent between runs. Examples of objects that don't satisfy those properties are sets and iterators over values of dictionaries.

Parameters:

- **params** (*iterable*) – an iterable of `torch.Tensor` s or `dict` s. Specifies what Tensors should be optimized.
- **defaults** (`dict[str, Any]`) – (dict): a dict containing default values of optimization options (used when a parameter group doesn't specify them).

```
optim.SGD([
    {'params': model.base.parameters(), 'lr': 1e-2},
    {'params': model.classifier.parameters()}
], lr=1e-3, momentum=0.9)

optim.SGD([
    {'params': model.base.named_parameters(), 'lr': 1e-2},
    {'params': model.classifier.named_parameters()}
], lr=1e-3, momentum=0.9)
```

Base class: optim.Optimizer()

문서: <https://docs.pytorch.org/docs/2.13/optim.html>

코드: <https://github.com/pytorch/pytorch/blob/v2.13.0/torch/optim/optimizer.py>